Appunti di Linguaggi e Tecnologie per il Web

Federico Falcone

April 9, 2026

Contents

1 Networking **3**
1.1 Topologie . . . 3
1.2 Standard . . . 3
1.2.1 Standard ISO-OSI . . . 4
1.3 Protocollo . . . 4

2 HTML **6**
2.1 Linguaggio di markup . . . 6
2.2 HyperText Markup Language . . . 6
2.3 Browser . . . 6
2.4 Struttura della pagina HTML . . . 7
2.4.1 Tag . . . 7
2.5 Struttura della pagina . . . 7
2.5.1 Testa . . . 7
2.5.2 Corpo . . . 7
2.6 Identazione . . . 7
2.7 Annidamento . . . 7
2.8 Commenti . . . 8
2.9 Maioscolo o minuscolo . . . 8
2.10 Testa (`<head>`) . . . 8
2.10.1 Titolo . . . 8
2.10.2 Metadati . . . 8
2.10.3 Meta Tag - Keywords . . . 8
2.10.4 Attributo HTTP-EQUIV . . . 8
2.11 Corpo (¡body¿) . . . 9
2.12 Tag interni al body . . . 9
2.12.1 Elementi inline vs block . . . 9
2.12.2 Testo - Tag predefiniti . . . 9
2.12.3 Struttura della pagina con tag `<div>` . . . 10
2.12.4 Testo - Stili . . . 11
2.12.5 Testo - Font . . . 11
2.12.6 Elenchi . . . 12
2.13 Ipertestualità (Link) . . . 13
2.13.1 Link . . . 13
2.13.2 Destinazioni . . . 13
2.13.3 Caratteristiche dei link . . . 13
2.13.4 Manipolazione dei link . . . 13
2.13.5 Percorsi dei link . . . 13
2.13.6 Link interni o ancore . . . 14
2.13.7 Colorare i link . . . 14
2.13.8 Immagini . . . 14
2.14 Tabelle (table) . . . 15
2.14.1 Bordi . . . 15
2.14.2 Celle collassate in una riga/colonna . . . 15
2.15 Form . . . 16
2.15.1 Input . . . 16
2.15.2 Action attribute . . . 17
2.15.3 Method attribute . . . 17

2.15.4 Name attribute . . . 17
2.15.5 Fieldset . . . 17
2.15.6 Altri attributi . . . 18
2.16 XHTML . . . 19

3 Bootstrap **19**
3.1 Come ottenere Bootstrap? . . . 19
3.1.1 CDN di Bootstrap . . . 19
3.2 Creare una pagina Web con Bootstrap . . . 20
3.3 Griglie Bootstrap . . . 21
3.4 Nuovo Flexbox grid system . . . 22
3.5 Classi di griglia . . . 23
3.5.1 Struttura di base di una griglia Bootstrap . . . 23
3.6 Tabelle Bootstrap . . . 23
3.7 Immagini Bootstrap . . . 23
3.8 Bottoni Bootstrap . . . 24
3.8.1 Elementi del pulsante Bootstrap . . . 24
3.8.2 Dimensioni dei pulsanti . . . 24
3.8.3 Pulsanti a livello di blocco . . . 24
3.8.4 Pulsanti attivi/disabilitati . . . 24
3.9 Personalizzazione . . . 24
3.10 Cards . . . 24
3.11 Nuovi helpers . . . 25
3.12 Introduzione a SASS . . . 25
3.12.1 Come funziona . . . 25
3.12.2 Caratteristiche di Sass . . . 26
3.12.3 Nesting . . . 26
3.12.4 Variabili . . . 27
3.13 Appunti utili . . . 27

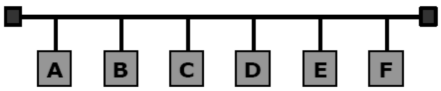
4 JavaScript **28**
4.1 Introduzione . . . 28
4.1.1 Applicazioni Web . . . 28
4.1.2 Applicazioni web: linguaggi . . . 28
4.1.3 Applicazioni web: linguaggi di programmazione e scripting . . . 28

1 Networking

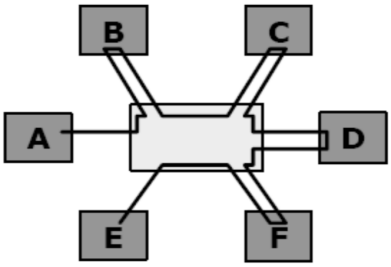
1.1 Topologie

Ogni tipologia di rete può essere fisicamente realizzata sfruttando topologie differenti:

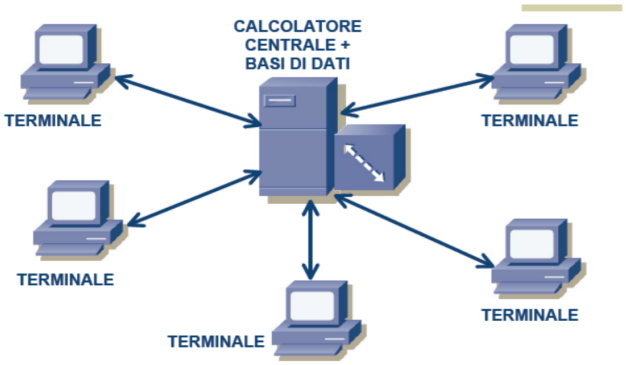
- topologia logica a Bus, cablata a stella

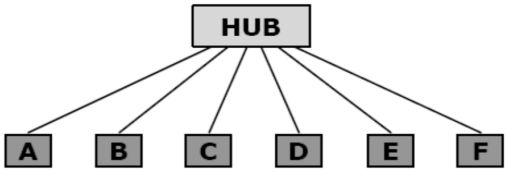


- topologia a stella



- centralizzata





1.2 Standard

La comunicazione tra nodi differenti e, possibilmente, basati su piattaforme hardware e/o software eterogenee necessita di standard. Storicamente si parla di standard di un produttore: scambio di informazioni possibile solo per sistemi della stessa marca (sistemi chiudi). Sono arrivati poi degli standard definiti da enti internazionali (livello di transizione dati).
Il vantaggio è che assicurano un vasto mercato per i dispositivi e il software. Inoltre consentono a prodotti provenienti da fornitori differenti di comunicare.
Di contro però è una tecnologia congelata e poi posono esserci più standard per la stessa funzione.

1.2.1 Standard ISO-OSI

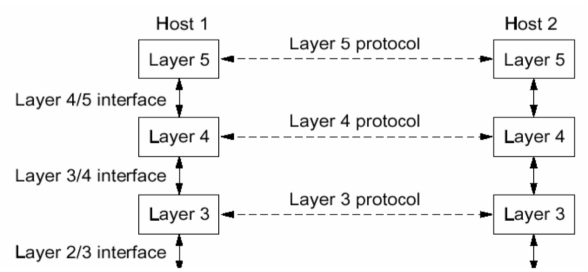
Lo standard ISO ha adottato un modello a strati. Il sistema di comunicazione viene diviso a strati ciascuno dei quali esegue una funzione predefinita. Strati ISO/OSI possono essere separati in due categorie

- funzioni dipendenti dalla rete (**Media layers**);
- funzioni orientate all'applicazione (**Host layers**).

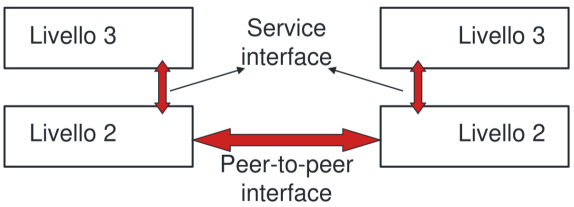
La funzione di ogni strato è codificata in una serie di regole e convenzioni usate per comunicare con lo strato remoto corrispondente (**protocollo**). Ogni strato fornisce dei servizi allo strato immediatamente superiore e utilizza inoltre dei servizi forniti dallo strato immediatamente inferiore.

1.3 Protocollo

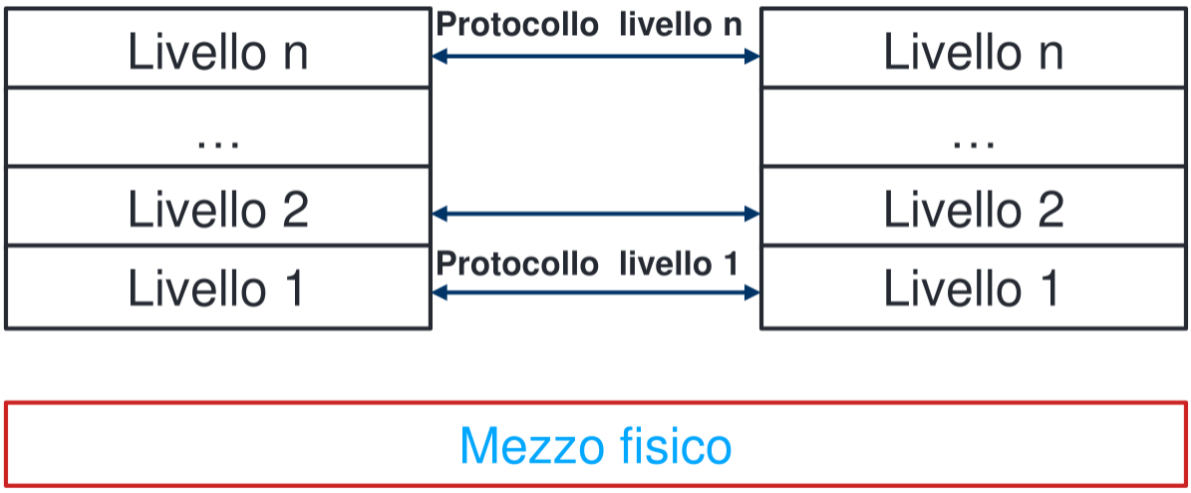
La comunicazione tra entità richiede cooperazione, ossia collaborazione per il conseguimento di uno scopo comune. Le comunicazioni sono regolate tramite protocolli.
Protocollo: insieme delle regole e convenzioni seguite da entità, dislocate su nodi distinti , che intendono comunicare per svolgere un compito comune. Tali regole hanno l'obiettivo di assicurare la cooperazione efficiente ed affidabile per la comunicazione tra nodi.
Sintassi: insieme e struttura dei comandi e delle risposte, formato dei messaggi.
Semantica: significato dei comandi, delle azioni, delle risposte da effettuare al momento della trasmissione e ricezione dei messaggi.
Temporizzazione: specifica delle possibilo sequenze temporali di emissione dei comandi e dei messaggi, nonchè eventuali risposte.
Ciascun protoccolo ha un'interfaccia "interna" verso il livello superiore o inferiore, e un'interfaccia "esterna" verso il livello equivalente di un altro modo.



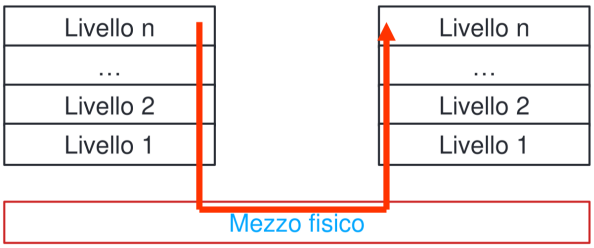
- **Service interface**("interna"):operazioni e servizi offerti al protocollo superiore.
- **Peer-to-peer interface**("esterna"): messaggi scambiati con un livello equivalente (peer) sull'altro nodo.



- Comunicazione logica tra *peer entity* (entità dello stesso livello).



- Comunicazione fisica (indiretta): la comunicazione tra *peer entity* è diretta solo a livello hardware.

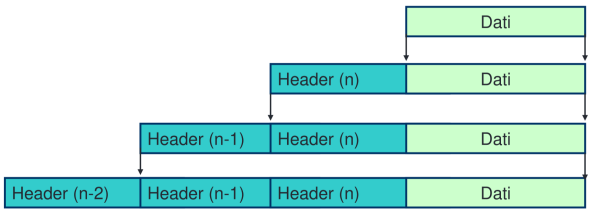


Un messaggio si compone di:

- **Header**: Protocol Control Information (PCI);
- **Dati**: Service Data Unit (SDU).

L'insieme delle due è chiamata **PDU** (Protocol Data Unit).

Incapsulamento del messaggio



La comunicazione avviene **logicamente** tra *peer*, ma in realtà attraversa tutti i livelli sottostanti, mediante **incapsulemnto del messaggio** a ciasun livello. Il sistema di comunicazione richiede un insieme di protocolli tra loro cooperanti (detti **protocol suite** o **protocol stack**).

2 HTML

2.1 Linguaggio di markup

Linguaggio di markup modellato per rendere esplicita una particolare interpretazione di un testo. Con il markup per sistemi informatici, specifichiamo le modalità esatte di utilizzo del testo nel sistema stesso. I linguaggi di markup sono i linguaggi più opportuni per strutturare e marcare i documenti in maniera indipendente dall'applicazione, favorendo la riusabilità, la flessibilità e la apertura ad applicazioni complesse.

2.2 HyperText Markup Language

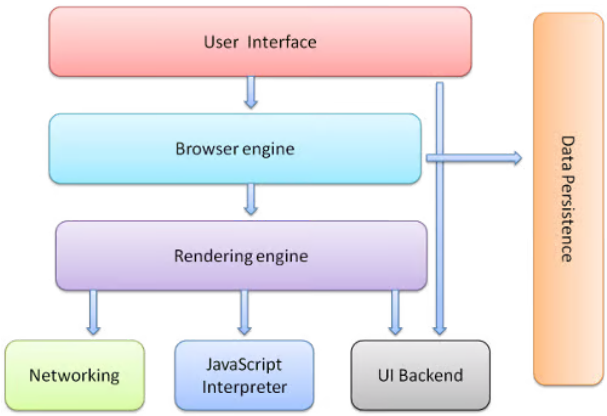
HTML è un linguaggio per la creazione di pagine Web.

- Standard per la rappresentazione.
- Descrive la struttura della pagina.
- Non è un linguaggio di programmazione, ma di markup.
- Descrive dati e regole su come mostrarli.
- Poche e semplici parole sintattiche.

2.3 Browser

È un programma per la navigazione Web. Richiede risorse attraverso il web (all'indirizzo richiesto) e la mostra nella finestra. Legge e inerpreta i documenti HTML, CSS, JavaScript, immagini sulla base di specifiche e standard.
Un Browser è formato da:

- **Layout Engine**: riceve input dal browser (URL bar, search box, mouse clicks and key presses) e li passa al rendering engine.
- **Rendering Engine**: riceve il codice HTML e lo interpreta visualizzandolo a video.
- **User interface**: controlli del browser, ad esempio, bottoni per andare avanti o indietro.
- **JavaScript engine**: engine che si prende cura di parsare ed eseguire codice JavaScript per poi ritornare il risultato dell'esecuzione.
- **Network Layer**: gestisce funzioni di rete, ad esempio, criptazione, richieste http, e tutte le configurazioni di rete.
- **Storage**: porzione di memoria dove il browser memorizza cached file, cookie, oggetti e dati create con JavaScript.
- **Operating System Interface**: componente che interagisce con il sistema operativo per disegnare diversi elementi e per gestire la finestra.



Il browser contiene anche tool per gli sviluppatori:

- Debug pagina.
- Analisi stili e sorgenti dei file.
- Visualizzazione HTTP request e response.
- Accesso a cookie e storage.
- Etc..

2.4 Struttura della pagina HTML

2.4.1 Tag

I tag sono marcatori che identificano porzioni di testo. Permettono la personalizzazione della pagina, e di identificare e processare le porzioni di testo contrassegnate da una certa marcatura.
Nome tag = nome funzione.
Il tag ha la seguente struttura

```
<tag attributi>contenuto</tag>
```

2.5 Struttura della pagina

Una pagina è formata da:

- **Riga di intestazione**: `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//IT">`. Indichiamo le specifiche W3C adottate. Esso dice al browser quale linguaggio è stato usato per il rendering.
- Il tag `<html>`: indica l'inizio di una pagina HTML. Tutto ciò compreso all'interno del tag `html` è il codice HTML.

  <html>
  ...
  </html>

- Documento HTML diviso in due:
  - Testa
  - Corpo

2.5.1 Testa

La testa contiene informazioni non immediate. Descrive come il documento deve essere letto e interpretato. Contiene meta-tag, script, stili.
È racchiuso nel tag `<head>`.

2.5.2 Corpo

Il corpo contiene il documento vero e proprio. Contiene tutti i tag per la realizzazione del sito Web.
È raachiuso nel tag `<body>`.

2.6 Identazione

Una buona norma è utilizzare caratteri di tabulazione. Aumenta la leggibilità, diminuendo così i tempi di modifica.

2.7 Annidamento

Una caratteristica di HTML sono i tag annidati.

2.8 Commenti

I commenti aumentano la leggibilità del codice. Una buona norma è commentare **solo** le parti significative. Permettono di orientarsi meglio all'interno del codice.

```
<!-- commento -->
```

2.9 Maioscolo o minuscolo

HTML è case unsensitive, mentre XHTML lo è. È consigliabile usare il carattere minuscolo.
Inoltre non è sensibile anche agli spazi e alle linee vuote.

2.10 Testa (`<head>`)

La testa contiene le informazioni/metadati relativi al documento. I browser di solito non presentano elementi che compaiono nella testa.

2.10.1 Titolo

L'elemento `<title>` permette di indicare il titolo della pagina.

2.10.2 Metadati

L'elemento `<meta>` definisce le informazioni relative alla pagina piuttosto che contenuto vero e proprio. Contiene 3 attributi principali:

- **Name**: contiene il nome della proprietà.
- **Content**: contiene il valore della proprietà.
- **http-equiv**: specifica le caratteristiche dell'HTTP responde header.

```
<META NAME="author" CONTENT="Federico">
```

2.10.3 Meta Tag - Keywords

I meta tag indicano alcune informazioni sul contenuto del sito. Sono separate da virgole, punto e virgola e spazio. Esse inoltre sono chiavi di ricerca.
Il motore di ricerca ricava le chiavi di indicizzazione. Bisogna fare attenzione nella scrittura delle keywords:

- evitare termini generici e particolari;
- riportare termini italiani ed inglese;
- singolare e plurale.

Evitare di ripetere le stesse parole e usare Keyword astute.

```
<meta name="keywords" content="calcio soccer tennis basket pallacanestro
    pallavolo volley inter milan juventus roma lazio...">
```

2.10.4 Attributo HTTP-EQUIV

Sono informazioni sulla comunicazione server-browser. Può essere utilizzato al posto dell'attributo name. L'attributo `http-equiv` fornisce un header HTTP per informazioni/valore dell'attributo content:

- Può essere usato per simulare un HTTP response header.
- Gli HTTP server usano questo attributo per ottenere informazioni circa l'HTTP response header.

Alcuni valori sono: `content-type, default-style, refresh`
Un esempio:

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
```

2.11 Corpo (¡body¿)

Il corpo di un documento contiene il contenuto vero e proprio. Il browser presenta il contenuto in diversi modi:

- Browser visuali: è una tela su cui disegnare testi, immagini, colori, ...
- Agenti audio: il contenuto stesso è parlato.

Esso contiene tutte le informazioni per la visualizzazione della pagina. Siccome gli style sheet sono il modo suggerito per specificare come verrà presentato un documento gli attributi del `body` sono stati deprecati.

2.12 Tag interni al body

2.12.1 Elementi inline vs block

Alcuni elementi HTML nel body sono "block-level", altri "inline". Gli elementi block-level possono contenere elementi inline e altri elementi block-level, mentre gli elementi inline contengono solo dati e altri elementi inline.

2.12.2 Testo - Tag predefiniti

I tag contenitori di testo sono block-level (ad eccezione di `<span>`).

- `<h1>,<h2>, ...` - heading;
- `<p>` - paragrafo;
- `<span>` - elemento inline per raggruppare porzioni di testo (ad esempio per colorarlo in maniera diversa rispetto al resto);
- `<div>` - è un contenitore degli elementi HTML.

Titoli - Heading
Sono previste sei grandezze predefinite per i titoli. Si va da `<h1>` che è la grandezza maggiore fino ad `<h6>` che è la grandezza minore.

```
<h1>Titolo</h1>
```

Paragrafo Il paragrafo è l'unità base di suddivisione del testo.

```
<p>Paragrafo</p>
```

`<p>` lascia una riga vuota prima e dopo il testo.

Blocco testo
`<div>` è un blocco contenitore. Esso va a capo ma non lascia righe di spazio.

```
<div>Blocco 1</div>
```

Non impone rappresentazioni di default.

Contenitore
È un contenitore generico e può essere annidato. È un elemento inline (continua sulla stessa riga del tag che lo contiene). Non va a capo.
Non impone rappresentazioni di default (a parte il fatto di essere un elemento inline).

Esempio

```
<h2>Titolo della pagina</h2>
<p>paragrafo 1</p>
<p>paragrafo 2</p>
<div>blocco 1</div>
<div>blocco 2
<span>contenitore 1</span>
<span>contenitore 2</span>
</div>
```

2.12.3 Struttura della pagina con tag `<div>`

`<div>` è uno tra i tag più utilizzati nella creazione di pagine Web, sopratutto nella creazione di layout. Esso fornisce un vero e proprio elemento strutturale della pagina. Suddivide gli spazi in zone per progettare il sito in modo semplice e dettagliato.
Molto utile quando usato insieme a fogli di stile.

Facciamo un esempio: immaginando di dover costruire un sito, pensiamo a come esso debba essere strutturato, tipicamente abbiamo:

- un contenitore (container);
- una parte alta (header);
- un corpo centrale (middle);
- un menù (navigation);
- una parte bassa (footer);
- spesso anche un pannello laterale (sidebar).

```
<div id="container">
<div id="header">
<div id="navigation"></div><!--#navigation-->
</div><!--#header-->
<div id="main"></div><!--#main-->
<div id="sidebar"></div><!--#sidebar-->
<div id="footer"></div><!--#footer-->
</div><!--#container-->
```

2.12.4 Testo - Stili

Sono possibili due tipi di stili:

- Fisici: definiscono lo stile grafico.
- Logici: forniscono informazioni sul ruolo svolto dal testo. Opzionalmente impongono uno stile grafico.

Stili fisici

- `<b>` bold: formatta il testo in **grassetto**;
- `<i>` italic: formatta il testo in *corsivo*;
- `<u>` underline: <u>sottolinea</u> il testo;
- `<strike>`: testo ~~barrato~~ (usato per correzioni);
- `<sup>` apice: $E = mc^2$;
- `<sub>` pedice: H_2O.

Stili logici

- `<abbr>` abbreviazione: non comporta nessun cambiamento grafico;
- `<address>` indirizzo;
- `<cite>` citazioni: testo in corsivo;
- `<samp>` esempio: testo a spaziatura fissa.

2.12.5 Testo - Font

Il carattere predefinito è il `Times New Roman`. Esso però è poco leggibile. Meglio `Verdana`, `Arial` o `Helvetica`.

```
<font face="verdana, arial, helvetica, sans-serif">testo</font>
```

È una buona norma utilizzare caratteri sicuri (sicuramente visualizzabili), non indicare un solo carattere e solo come ultima spiaggia sarà usato il `Times New Roman`.
Per scegliere il colore del testo:

```
<font color="#0000FF">testo blu</font>
```

LA scelta del colore e del carattere può essere fatta nello stesso tag.

```
<font color="#0000FF" face="verdana">testo blu</font>
```

Font annidati

```
<font color="blue" face="verdana">
    testo blu
    <font color="red">
        testo rosso
    </font>
</font>
```

Dimensione del testo

Si utilizza l'attributo `size` del tag `font`. Ci sono 2 modi per impostare la dimensione:

- Valore tra 1 e 7.

  <font size="3">Dimensione 3</font>

- Valori relativi alla dimensione del font.

  <font size="+2">Dimensione +2</font>

  Utilizza la dimensione del font di base. Il tag `<basefont>` permette di cambiare il font di base.

2.12.6 Elenchi

Gli elenchi possono essere:

- **ordinati**;
- **non ordinati**;
- **di definizioni**.

Sintassi:

```
<elenco>
    <elemento> primo elemento
    <elemento> secondo elemento
</elenco>
```

La chiusura dell'elemento è opzionale. La sintassi degli elenchi di definizione è leggermente diversa.

Elenchi ordinati
Gli elementi sono numerati progressivamente.
Il tag dell'elenco ordinato è `<ol>` (ordered list), mentre quello dell'elemento è `<li>` (list item).

```
<div>Titolo</div>
<ol>
    <li>Primo elem prima tabella
    <li>Secondo elem prima tabella
    <ol>
        <li> Primo elem seconda tabella
        <li> Secondo elem seconda tabella
    </ol>
</ol>
```

Elenchi non ordinati
Gli elementi non sono numerati. Si ottiene quindi un elenco puntato. Il tag è `ul` (unordered list).

```
<ul>
    <li> Primo elemento
    <li> Secondo elemento
    <ul>
        <li> Primo elemento
        <li> Secondo elemento
    </ul>
</ul>
```

Elenco di definizioni
Per creare un elenco di definizione si utilizza il tag `dl` (definition list) e il termine da definire `<dt>` (definition term). Per la definizione del termine si utilizza il tag `<dd>` (definition description).

```
<p> Titolo
<dl>
    <dt>&ltp&gt
        <dd>apertura paragrafo
    <dt>&ltdiv&gt
        <dd>apertura blocco di testo
    <dt>&ltspan&gt
        <dd>apertura elemento inline
    Altri tag...
</dl>
<dl>
```

2.13 Ipertestualità (Link)

2.13.1 Link

I link sono formati da due componenti:

- il contenuto (testo o immagine) che nasconde il collegamento;
- la risorsa puntata.

```
<a href="indirizzo"> link </a>
```

La testa del link è *link*, mente la coda è *indirizzo*.
La coda può essere una pagina HTML, un'immagine, documenti, altri file.

2.13.2 Destinazioni

Le destinazioni dei link possono essere:

- Pagina HTML: aperta nel browser.
- Immagine (.gif, .jpeg): aperta nel browser.
- Altri documenti (.doc, .pdf):
  - visualizzata nel browser;
  - richiesta di salvataggio.
- Altri file (.zip, .exe): richiesta di salvataggio.

2.13.3 Caratteristiche dei link

I link possono avere diversi stati:

- link a riposo: blu;
- link visitato: violetto;
- link attivo: passaggio da una pagina all'altra. Utile nel passato.
- link al passaggio del mouse: solo con fogli di stile.

2.13.4 Manipolazione dei link

- `<body link="red">`: cambia il colore dei link;
- `<body vlink="green">`: cambia il colore dei link visited;
- `body alink="yellow">`: cambia il colore dei link attivi.

2.13.5 Percorsi dei link

I percorsi dei link possono essere:

- assoluti: viene indicato il percorso per esteso;
- relativi: fanno riferimento alla posizione del file a cui si riferisce il link a partire dalla posizione della pagina che stiamo sviluppando.

Percorsi assoluti

```
<a href="http://www.miosito.it/cartella/miofile.html"> qui</a>
```

Percorsi relativi

```
<a href="sottodir/sottolink.html">Clicca qui</a>
```

2.13.6 Link interni o ancore

Permette di creare un indice.

- Ancora

```
    <a name="primo">ancora</a>
```

- Riferimento all'ancora

```
    <a href="#primo"> vai all'ancora </a>
```

- Riferimento ad inizio pagina

```
    <a href='#'>torna su</a>
```

2.13.7 Colorare i link

Per colorare solo un link:

```
<a href="...">
    <font color="green">
        link verde
    </font>
</a>
```

2.13.8 Immagini

Il web non è solo ipertesto, ma bensì ipermedia.

```
<img src="pic.jpg" alt="description" width="128"height="128">
```

- Attributo `src`: sorgente del file (percorso di memorizzazione file).
- Attributo `alt`: descrizione mostrata al passaggio sull'immagine.
- Attributi `width` e `height`: dimensioni dell'immagine.

Immagine come link:

```
<a href="destinazione.html">
    <img src="pic.jpg">
</a>
```

2.14 Tabelle (table)

Le tabelle si definiscono tramite l'elemento `<table>`. Ogni riga è definita con il tag `<tr>` contenuto in `table`. Ogni cella è definita con il tag `<td>` contenuto in `<tr>`.
Una riga può essere anche divisa in table heading usando l'elemento `<th>`. Di solito sono centrari e in grassetto.
Elemento `<caption>` serve per definire la caption della tabella.

```
<table style="width:100%">
    <tr>
        <td>Jill</td>
        <td>Smith</td>
        <td>50</td>
    </tr>
    <tr>
        <td>Eve</td>
        <td>Jackson</td>
        <td>94</td>
    </tr>
    <tr>
        <td>John</td>
        <td>Doe</td>
        <td>80</td>
    </tr>
</table>
```

2.14.1 Bordi

Per modificare i bordi della tabella si utilizza l'attributo `border`. Se non specificato nessun bordo:

```
<table border="1" style="width:100%">
```

2.14.2 Celle collassate in una riga/colonna

Si utilizza l'attributo `rowsspan` per collassare un riga:

```
<th rowspan="2">Telephone</th>
<td rowspan="2">Telephone</td>
```

Si utilizza l'attributo `colspan` per collassare una colonna:

```
<th colspan="2">Telephone</th>
<td colspan="2">Telephone</td>
```

2.15 Form

I form sono utilizzati per collezionare gli input degli utenti. È basato sull'elemento `<form>`. Esso specifica diversi elementi.

2.15.1 Input

È l'elemento più importante. Sono presenti diverse varianti a seconda da come si indica l'attributo `type`:

- `text`: definisce input testo normale;
- `radio`: definisce un input radio button (per selezionare una tra diverse scelte);
- `submit`: definisce un bottone submit (per sottomettere il form).

Text input
L'input text definisce un campo di testo di una linea. `<input type="password">` è un tipo speciale di campo testo dove il valore inserito viene nascosto.

```
<form>
    First name:<br>
    <input type="text" name="firstname">
    <br>
    Last name:<br>
    <input type="text" name="lastname">
</form>
```

Text area
Il text area definisce un campo testo a linee multiple.

```
<textarea name="message" rows="10" cols="30">
    The cat was playing in the garden.
</textarea>
```

Radio button
Il radio button definisce un input che permette **una** scelta tra diverse opzioni.

```
<form>
    <input type="radio" name="sex" value="male" checked>Male
    <br>
    <input type="radio" name="sex" value="female">Female
</form>
```

Checkbox input
La checkbox input definisce una serie di checkbox che permettono la scelta **multipla** tra diverse opzioni.

```
<form>
    <input type="checkbox" name="vehicle1" value="Bike"> I have a bike
    <br>
    <input type="checkbox" name="vehicle2" value="Car"> I have a car
</form>
```

Drop-down list
La drop down-drown list definisce un menù a tendina. L'elemento `<option>` definisce le possibili opzioni.

```
<select name="cars">
    <option value="volvo">Volvo</option>
    <option value="saab">Saab</option>
    <option value="opel">Opel</option>
    <option value="audi">Audi</option>
</select>
```

Submit button
La submit button definisce un bottone per sottomettere la form ad un handler. L'handler è una pagina lato server che processa i dati del form. L'handler è definito con l'attributo `action`.

```
<form action="action_page.php">
    First name:<br>
    <input type="text" name="firstname" value="Mickey">
    <br>
    Last name:<br>
    <input type="text" name="lastname" value="Mouse">
    <br><br>
    <input type="submit" value="Submit">
</form>
```

Permette anche di eseguire semplici azioni:

```
<button type="button" onclick="alert('Hello World!')">Click Me!</button>
```

2.15.2 Action attribute

L'attributo `action` definisce le azioni che devono essere fatte quando il form è inviato. La sottomissione viene comunque effettuata con il bottone submit. La pagina viene inviata a una pagina web lato server. Ad esempio uno script lato server gestisce la form inviata. Se l'attributo action è omesso, si considera la pagina corrente di default.

2.15.3 Method attribute

L'attributo `method` specifica l'operazione HTTP (GET o POST) da usare per inviare la form.
GET, è l'opzione di default, viene usata per l'invio di form passive e senza informazioni sensibili. I dati inviati sono visibili nell'indirizzo. È ottima per piccola quantità di dati.
GET è quindi utilizzata per l'invio di form che aggiornano dati, mentre POST fornisce maggiore sicurezza perchè i dati non sono visibili nell'indirizzo.

2.15.4 Name attribute

Ogni campo input deve avere un nome.

```
<form action="action_page.php">
    First name:<br>
    <input type="text" name="firstname" value="Mickey">
    <br>
    Last name:<br>
    <input type="text" name="lastname" value="Mouse">
    <br><br>
    <input type="submit" value="Submit">
</form>
```

2.15.5 Fieldset

Per raggruppare campi di una form, si utilizza `<fieldset>`. Si può definire anche una caption per la form `<legend>`.

```
<form action="action_page.php">
    <fieldset>
        <legend>Personal information:</legend>
        First name:<br>
        <input type="text" name="firstname" value="Mickey">
        <br>
        Last name:<br>
        <input type="text" name="lastname" value="Mouse">
        <br><br>
        <input type="submit" value="Submit">
    </fieldset>
</form>
```

2.15.6 Altri attributi

- `value`: definisce il valore iniziale;
- `readonly`: specifica che il valore non può essere cambiato;
- `disabled`: specifica un campo disabilitato e non utilizzabile (non viene inviato);
- `size`: specifica la dimensione in caratteri del campo;
- `maxlength`: specifica la dimensione massima del contenuto del campo.

2.16 XHTML

XHTML è quasi identito ad HTML, ma è più stringente. È supportato dai principali browser.

Perchè utilizzare XHTML
Molte pagine Web contengono HTML errato. Il seguente codice viene visualizzato bene nella maggior parte dei casi anche se non segue le regole di HTML:

```
<html>
    <head>
        <title>This is bad HTML</title>
    </head>
    <body>
        <h1>Bad HTML</h1>
        <p>This is a paragraph</p>
    </body>
</html>
```

Diversi browser che eseguo su device diversi richiedono maggiori risorse o potenza coputazionale per interpretare un markup sbagliato.
XHTML combina HTML e XML.

3 Bootstrap

Web design reattivo
Il responsive web design riguarda la creazione di siti web che si adattano automaticamente per avere un bell'aspetto su tutti i dispositivi, dai piccoli telefoni ai desktop di grandi dimensioni.
Bootstrap è il framework HTML, CSS e JavaScript più popolare per lo sviluppo di siti Web responsive e mobile-first.

Bootstrap è un framework front-end gratuito per uno sviluppo web più rapido e semplice. Bootstrap include modelli di progettazione basati su HTML e CSS per tipografia, moduli, pulsanti, tabelle, navigazione, modali, caroselli di immagini e molti altri, oltre al plug-in JavaScript opzionali.
Bootstrap dà anche la possibilità di creare facilmente design reattivi.

3.1 Come ottenere Bootstrap?

Ci sono due modi per iniziare ad usare Bootstrap:

- scaricare Bootstrap da `getbootstrap.com` e seguire le istruzioni da lì;
- includere Bootstrap da un CDN (Content Delivery Network).

3.1.1 CDN di Bootstrap

Bisogna includere i seguenti CSS, Javascript e jQuery di Bootstrap da MaxCDN nella pagina web:

```
<!-- Ultimo CSS Bootstrap compilato e minimizzato -->
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css
    /bootstrap.min.css">
```

```
<!-- JavaScript Bootstrap piu recente compilato -->
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js
    "></script>
```

```
<!-- ultima libreria jQuery -->
<script src="https://code.jquery.com/jquery-latest.js"></script>
```

Vantaggio dell'utilizzo di Bootstrap CDN: molti utenti hanno già scaricato Bootstrap da MaxCDN visitando un altro sito. Di conseguenza, verrà caricato dalla cache quando visitano il sito, il che porta a tempi di caricamento più rapidi. Inoltre, la maggior parte dei CDN farà in modo che una volta che un utente richiede un file da esso, verrà servito dal server più vicino a loro, il che porta anche a tempi di caricamento più rapidi.

3.2 Creare una pagina Web con Bootstrap

Bisogna aggiungere il doctype HTML5 siccome Bootstrap utilizza elementi HTML e proprietà CSS di questo doctype.

```html
<!doctype html>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Bootstrap demo</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH" crossorigin="anonymous">
    </head>
    <body>
        <h1>Hello, world!</h1>
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous"></script>
    </body>
</html>
```

Bootstrap è un mobile-first:

- Bootstrap 3 è progettato per rispondere ai dispositivi mobili. Gli stili mobile-first fanno parte del frameword principale.
- Per garantire un rendering e uno zoom al tocco corretti, bisogna aggiongere il tag `<meta>` all'interno dell'elemento `<head>`.

```html
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

- La parte `width=device-width` imposta la larghezza della pagina in modo che segue la larghezza dello schermo del dispositivo (che varierà a seconda del dispositivo).
- La parte `initial-scale=1` imposta il livello di zoom iniziale quando la pgina viene caricata per la prima volta dal browser.

Contenitori
Bootstrap richiede anche un elemento contenitore per racchiudere i contenuti del sito. Ci sono due classi di contenitori tra cui scegliere:

- La classe `.container` fornisce un **contenitore a larghezza fissa reattivo**.
- La classe `.container-fluid` fornisce un **contenitore a larghezza intera**, che copre l'intera larghezza del viewport.

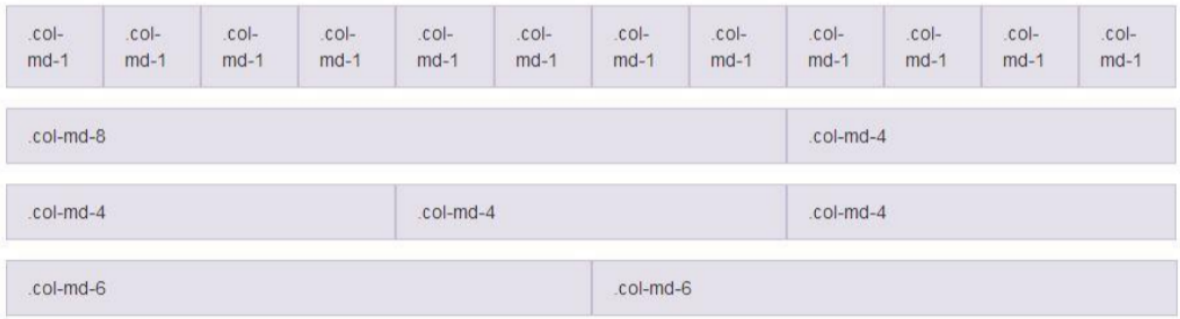
Nota: i contenitori non sono modificabili (non è possibile inserire un contenitore all'interno di un altro contenitore).

3.3 Griglie Bootstrap

Il sistema a griglia di Bootstrap consente fino a 12 colonne nella pagina. Se non si desira utilizzare tutte e 12 le colonne singolarmente, si possono raggruppare insieme per creare colonne più larghe.

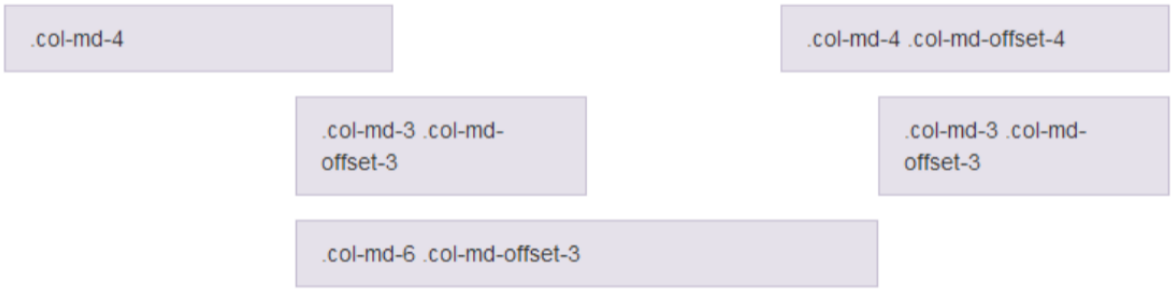
```
<div class="col-md-12">Si estende su 12 colonne</div>
<div class="col-md-6">Intervallo 6</div><div class="col-md-6">Intervallo 6</div>
<div class="col-md-4">Intervallo 4</div><div class="col-md-8">Intervallo 8</div>
<div class="col-md-4">Intervallo 4</div><div class="col-md-4">Intervallo 4</div>
    <div class="col-
md-4">Intervallo 4 </div>
```

Il sistema a griglia di Bootstrap è reattivo e le colonne si riorganizzeranno automaticamente a seconda delle dimensioni dello schermo.
È indivisuato quindi dalla classe `.col-(nome breakpont)-*`



È possibile innestare griglie nelle colonne creando delle `.row` nelle colonne, reiterando il meccanismo.

È possibile gestire i salti di colonna con gli offset `col-md-offset-*`.



Di default è possibe gestire 5 dimensioni del viewport:

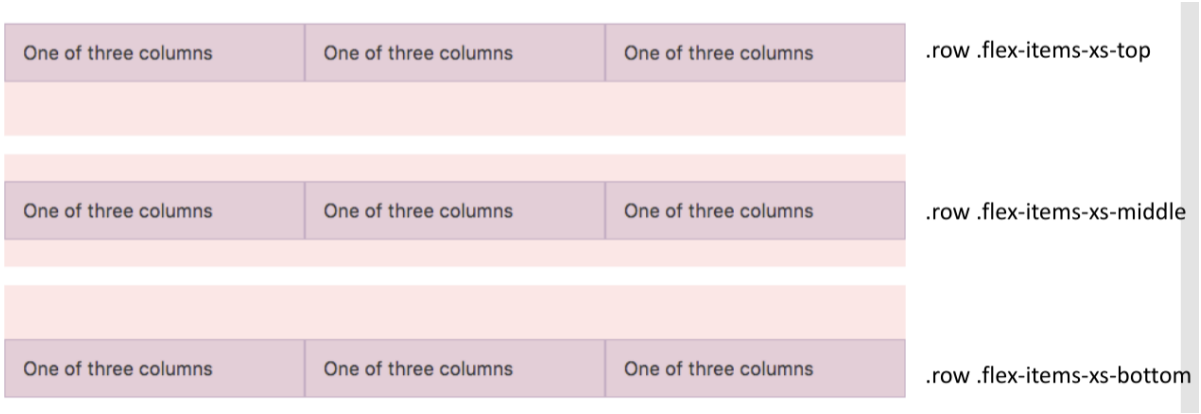
- Extra small (`.col-xs-*`), per gli smartphone.
- Small (`.col-sm-*`), per i tablet.
- Medium (`.col-md-*`), per i desktop.
- Large (`.col-lg-*`), per i desktop larghi.
- Extra large (`.col-xl-*`), per i desktop molto larghi.

3.4 Nuovo Flexbox grid system

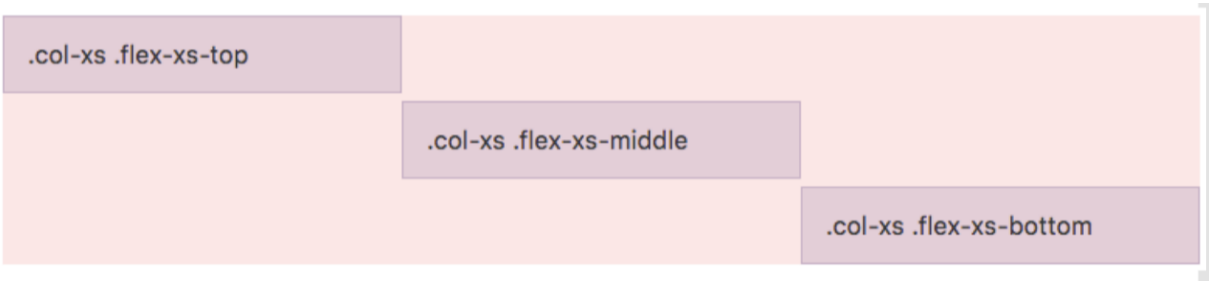
Con il nuovo Grid System non dobbiamo più indicare il numero nella classe della colonna, basterà usare solo la classe `.col-(nome breakpoint)` e si adatteranno in automatico allo spazio disponibile.

```
<div class="row">
    <div class="col-xs"> 1 of 3 </div>
    <div class="col-xs"> 2 of 3 </div>
    <div class="col-xs"> 3 of 3 </div>
</div>
```

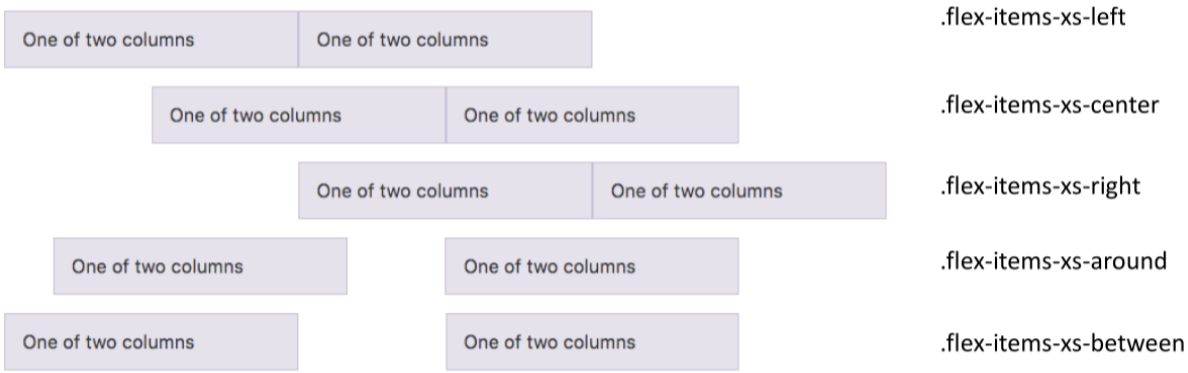
Possiamo allineare le righe in verticale:



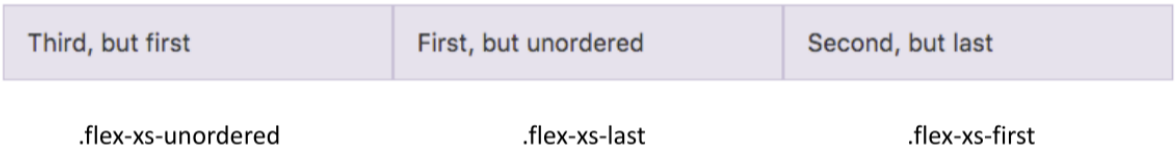
... e anche le colonne



Riordino orizzontale



Riordino della posizione



3.5 Classi di griglia

Il sistema di griglia di Bootstrap ha quattro classi:

- xs;
- sm;
- md;
- lg.

Le classi di cui sopra possono essere combinate per creare layout più dinamici e flessibili.

3.5.1 Struttura di base di una griglia Bootstrap

```
<div class="row">
    <div class="col-*-*"></div>
</div>
<div class="row">
    <div class="col-*-*"></ div>
    <div class="col-*-*"></div>
    <div class="col-*-*"></div>
</div>
<div class="row">
...
< /div>
```

Primo; creare una riga (`<div class="row"`). Quindi, aggiungere il numero desiderato di colonne (tag con classi `.col-*-*` appropriate). Nota che in numeri in `.col-*-*` dovrebbero sempre sommare fino a 12 per ogni riga.

3.6 Tabelle Bootstrap

Un tavolo Bootstrap di base ha una **leggera imbottitura e solo divisori orizzontali**. La classe `.table` aggiunge uno stile di base a una tabella:

- **righe a strisce**: la classe `.table-striped` aggiunge strisce zebrate a una tabella.
- **Tavolo bordato**: la classe `.table-bordered` aggiunge bordi su tutti i lati della tabella e delle celle.
- **Righe al passaggio del mouse**: la classe `.table-hover` abilita uno stato hover sulle righe della tabella.
- **Tabelle reattive**: la classe `.table-responsibe` crea una tabella reattiva. La tabella scorrerà quindi orizzontalmente su dispositivi di piccole dimensioni (768 px). Quando si visualizza su qualcosa di più largo di 768 px, non c'è differenza.

3.7 Immagini Bootstrap

- **Angoli arrotondati**: la classe `.img-rounded` aggiunge angoli arrotondati a un'immagine.
- **Cerchio**: la classe `.img-circle` modella l'immagine in un cerchio.
- **Miniatura**: la classe `.img-thumbnail` modella l'immagine in una miniatura.
- **Immagine reattive**: le immagini sono disponibili in tutte le dimensioni. Così fanno gli schermi. Le immagini reattive si adattano automaticamente alle dimensioni dello schermo. Crea immagine reattive aggiungendo una classe `.img-responsive` al tag `<img>`. L'immagine verrà quindi ridimensionata in modo corretto rispetto all'elemento genitore. La classe `.img-responsive` applica `display: block`; e larghezza massima: 100%: e altezza: `auto`; all'immagine.

3.8 Bottoni Bootstrap

Stili dei pulsanti: Bootstrap fornisce 7 stili di pulsanti con le seguenti classi:

- `.btn-predefinito`;
- `.btn-primary`;
- `.btn-success`;
- `.btn-info`;
- `.btn-warning`;
- `.btn-danger`;
- `.btn-link`.

3.8.1 Elementi del pulsante Bootstrap

Le classi di pulsanti possono essere utilizzaate sui seguenti elementi:

- `<a>`;
- `<pulsante>`;
- `<input>`.

3.8.2 Dimensioni dei pulsanti

Bootstrap fornisce quattro dimensioni di pulante con le seguenti classi:

- `.btn-lg`;
- `.btn-md` (dimensione predefinita);
- `.btn-sm`;
- `.btn-xs`.

3.8.3 Pulsanti a livello di blocco

Un pulsante a livello di blocco copre l'**intera larghezza** dell'elemento genitore. Per fare ciò si aggiunge la classe `.btn-block`.

3.8.4 Pulsanti attivi/disabilitati

Un pulsante può essere impsotato su uno stato attivo (appare premuto) o disabilitato (non cliccabile): la classe `.active` fa apparire premuto un pulsante e la classe `.disabled` rende un pulsante non cliccabile.

3.9 Personalizzazione

Il codice sorgente di Bootstrap può essere personalizzato:

- Attraverso un tool online (V3).
- Less (V3).
- Sass (V3 e V4).

3.10 Cards

Wells, thumbnails e panels ci lasciano in favore delle cards.

3.11 Nuovi helpers

Nuovi classi di supporto per gestire margin, padding e proprietà display

- `.pb-0` (padding-bottom: 0;);
- `-p-3` (padding: 3rem;);
- `mt-0` (margin-top: 0;);
- `.ml-1` (margin-left: 1rem;);
- `.mx-auto` (margin-left: auto; margin-right: auto;);

Migliorate le classi di supporto per allineare il testo in base al breakpoint:

- `text-xs-left` (-right, -center);
- `text-sm-left` (-right, -center);
- `text-md-left` (-right, -center);
- `text-lg-left` (-right, -center);
- `text-xl-left` (-right, -center);

Migliorate le classi di supporto per allineare gli elementi in base al breakpoint:

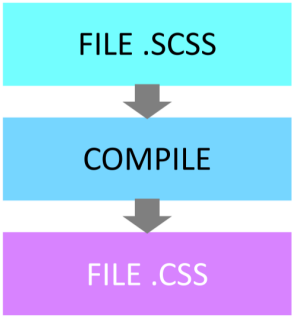
- .float-xs-left (-right);
- .float-sm-left (-right);
- .float-md-left (-right);
- .float-lg-left (-right);
- .float-xl-left (-right);

Migliorate le classi per nascondere o mostrare elementi in base al breakpoint: `.hidden-xs` diventa `.hidden-xs-up`.
Ad esempio: `.hidden-md-up` nasconderà tutti gli elementi dal breakpoint md in su.

3.12 Introduzione a SASS

SASS: è un **pre-processore CSS**, un meta-linguaggio che ci permette di usare una sintassi più simile a quella dei linguaggi di programmazione offrendo moltissimi vantaggi tra cui una **maggiore manutenibilità del codice**. SASS è nato per rimediare a disordine e ripetizioni, per introdurre una strututra formale dei CSS, migliorare la scrittura del codice e renderlo più manutenibile e ottimizzare la gestione delle media queries e andare incontro alle esigenze del mercato in crescita di dispositivi mobili.

3.12.1 Come funziona



3.12.2 Caratteristiche di Sass

- **Nesting**: possibilità di nidificare i selettori ottenendo un codice pulito.
- **Variabili**: come in altri linguaggi è un insieme di dati modificabili istuati in una porzione di memoria;
- **Importazione**: permette di importare altri file `scss` o `css` senza generare un grosso numero di richieste HTTP che rallenterebbero inevitabilmente il sito.
- **Funzioni**: in Sass esistono dei set di funzioni preconfezionate per effettuare diverse operazioni, è inoltre possibile sviluppare funzioni personalizzate;
- **Mixin**: permette di realizzare dei set riutilizzabili di regole CSS;
- **Logica condizionale**: @if, @else, @else if.
- **Cicli**: @for, @each, @while.
- **Compass, scount, ecc...**: framework opensource che offrono un'ampia gamma di mixin, funzioni e variabili preconfezionati.

3.12.3 Nesting

Con il **nesting** basta dichiarare una sola volta il nome dell'elemento radice per inestare le altre regole figlie all'interno.

```
#box{
    background-color: red;

    header{
        background-color: yellow;

        h1{ color: black; }
    }

    section h1{ color: blue; }
}
```

```
#box{
    background-color: red;
}

#box header{
    background-color: yellow;
}

#box header h1{
    color: black;
}

#box section h1{
    color: blue;
}
```

SASS fornisce il selettore speciale "**&**" che, utilizzato all'interno di un **nesting** farà riferimento all'elemento radice.

```
#box {

    &-main {
        width: 100%
    }
    &.small {
        width: 100%
    }
}
```

```
1  #box-main {
2      width: 100%;
3  }
4  #box.small {
5      width: 100%;
6  }
7
8
9
```

"&" può essere utilizzato per definire le pseudoclassi di un elemento.

```
a {
    color: #000;
    &:hover{
    color: #aaa;
    }
}
```

```
1  a {
2      color: #000;
3  }
4  a:hover {
5      color: #aaa;
6  }
7
```

3.12.4 Variabili

Variabili: come nei linguaggi di programmazione il concetto di variabile è pressoché identico, una variabile **SASS** è un contenitore virtuale di:

- stringhe di testo;
- valori numerici;
- numeri in virgola mobile;
- valori booleani.

Ad esempio possiamo creare dei set di colori:

```
$base-color: #abcdef;

button {
    background: $base-color;
}
a {
    color: $base-color;
}
header {
    border-bottom: 5px;
    border-color: $base-color;
}
```

```
button {
  background: #abcdef;
}

a {
  color: #abcdef;
}

header {
  border-bottom: 5px;
  border-color: #abcdef;
}
```

Oppure gestire facilmente le media queries.

3.13 Appunti utili

- Se voglio fare in modo che un testo sia colorato di un certo colore, utilizzare la classe `.text-white`.
- Per dare un spazio tra un elemento e l'altro, utilizzare le classi di **spacing**.
- Al posto di `<small>` si può usare la classe `.small`.
- Utilizzare `<span>` per raggruppare un testo e applicare uno stile specifico.
- Per cambiare il colore si può usare anche la classe `.text-*colore*`. Aggiungedoci -*valore* si aggiunge la trasparenza.
- Per modificare font testo, utilizzare `.fw-fold`, ad esempio.
- Per creare un testo con la sidebar, utilizzare `overflow-auto`.
- Per far sì che una immagine entri correttamente in un contenitore, utilizzare la proprietà CSS `object-fit: cover;`.

4 JavaScript

4.1 Introduzione

4.1.1 Applicazioni Web

Le risorse in rete identificata tramite URL è di 3 tipi:

- pagina **statica** (HTML) il cui contenuto è **fisso**, definito nel momento in cui la pagina è stata scritta;
- pagina **dinamica** il cui contenuto viene generato (selezionato, composto) al momento della richiesta (o della visualizzazione);
- **applicazione web**: software sviluppato e utilizzato attraverso attraverso tecnologie web;
  - linguaggi di mark-up, scripting e programmazione;
  - teconologie client-side e server-side;

4.1.2 Applicazioni web: linguaggi

- linguaggi di mark-up descrivono documenti strutturati (contenuto + struttura).
- linguaggi di programmazione descrivono programmi come sequenza di istruzioni;
- linguaggi di scripting, tipo particoalre di linguaggio di programmazione, per scrivere script.

4.1.3 Applicazioni web: linguaggi di programmazione e scripting

Un programma è un insieme di istruzioni, mentre i linguaggi di programmazione sono linguaggi per scrivere programmi. Un linguaggio è composto da:

- **Alfabeto**: insieme di simboli con cui si possono costruire i termini del linguaggio (lessico).
- **Sintassi**: definita da una grammatica che fornisce regole di composizione dei termini in frasi bene formate